

1. softmax 的 max trick

softmax:

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

问题在于 $\exp$ 很容易溢出。比如 $z = [1000, 1001, 1002]$ ，直接算 $\exp(1000)$ 、 $\exp(1001)$ 、 $\exp(1002)$ ，浮点数会先炸掉，后面的归一化也就没意义了。

做法是令：

$$m = \max_j z_j$$

然后改算：

$$\text{softmax}(z)_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}$$

这个变形不改变结果：

$$\begin{aligned} \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)} \exp(z_j - m) &= \frac{\exp(z_i) \exp(-m)}{\sum_j \exp(z_j) \exp(-m)} (\exp(z_j) \exp(-m)) \\ &= \frac{\exp(z_i) \exp(-m)}{\exp(-m) \sum_j \exp(z_j)} = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \end{aligned}$$

所以本质上只是给所有 logits 同时减去一个常数。softmax 对这种整体平移不敏感。

为什么选最大值？因为减完之后：

$$z_i - m \leq 0$$

于是：

$$0 < \exp(z_i - m) \leq 1$$

最大的那个元素变成 $\exp(0) = 1$ ，其他元素都不超过 1，因此避免了上溢。

例子：

$$[1000, 1001, 1002] - 1002 = [-2, -1, 0]$$

所以实际算的是：

$$[\exp(-2), \exp(-1), \exp(0)] \approx [0.1353, 0.3679, 1]$$

归一化后：

$$[0.0900, 0.2447, 0.6652]$$

这和原来的 softmax 数学结果一样，只是中间值不会溢出。

代码就是：

```
import numpy as np

def softmax(x, axis=-1):
    x = np.asarray(x)
```

```
x = x - np.max(x, axis=axis, keepdims=True)
e = np.exp(x)
return e / np.sum(e, axis=axis, keepdims=True)
```

顺便说一句：减最大值可能让极小项下溢成 0，但这通常可以接受；上溢成 `inf` 再进入归一化更危险，容易得到 NaN。